车牌追踪

1.开篇说明

本案例使用到的电机驱动板是亚博的四路电机驱动板，如果使用该驱动板，请先阅读四路电机驱动板资料中的《电机介绍以及用法》，了解清楚自己现使用的电机参数、接线方式、供电电压。以免造成烧坏主板或者电机的后果。

2.实验准备

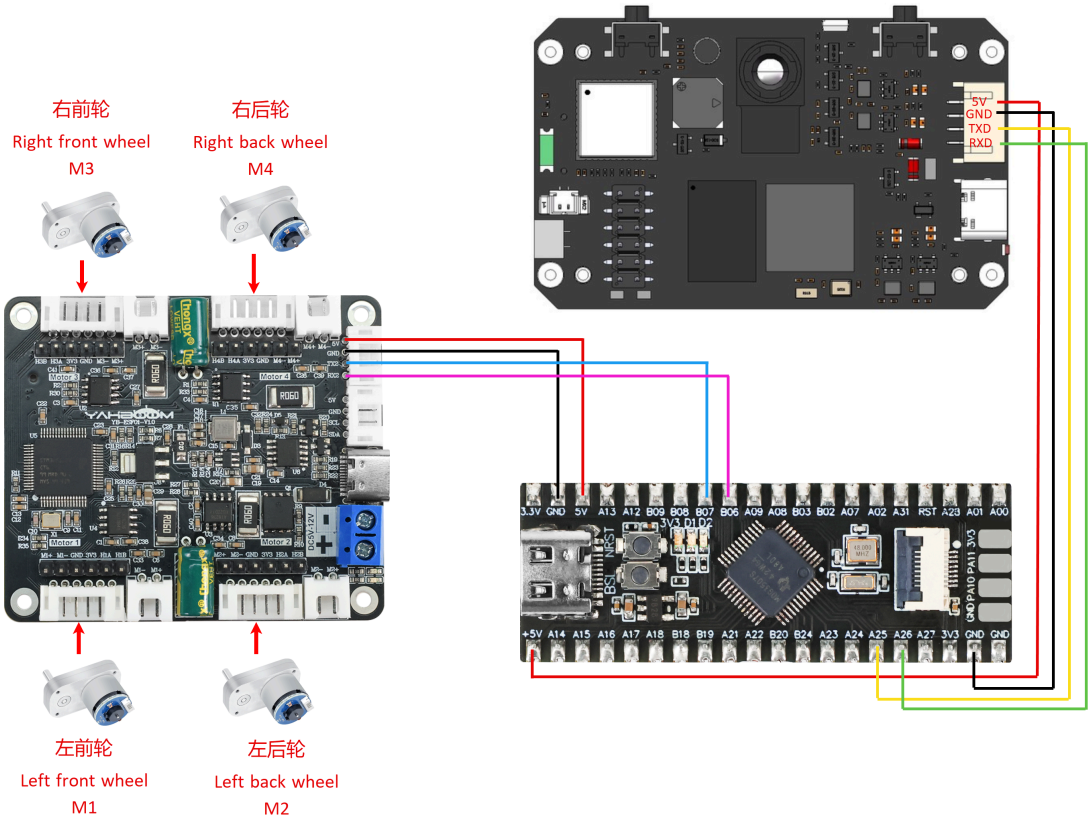
轮式小车底盘、4*L520电机、12V锂电池、K230视觉模块、亚博MSPM0G3507核心板。

4个电机接口对应小车的关系如下：

M1 -> 左上电机(小车的左前轮)　M3 -> 右上电机(小车的右前轮)

M2 -> 左下电机(小车的左后轮)　M4 -> 右下电机(小车的右后轮)

硬件接线



接线引脚

四路电机驱动板	亚博MSPM0G3507
5V	5V

四路电机驱动板	亚博MSPM0G3507
GND	GND
RX2	PB6
TX2	PB7

下面以M1电机为例，其他电机依此类推：

电机	四路电机驱动板(Motor)
M2	M1-
VCC	3V3
A	H1A
B	H1B
GND	GND
M1	M1+

K230	亚博MSPM0G3507
5V	5V
GND	GND
TXD	PA25
RXD	PA26

3.实验操作

1.按照教程的硬件接线说明进行接线。

2.打开MSPM0G3507使用的ProtocolHub工程，首先查看main.c文件的开头部分，第一个要将MOTOR_TYPE的数字修改为自己正在使用的电机的，修改正确之后就编译加烧录，最后reset或者重新上电。

3.将License_Track.py放置于K230的sdcard根目录下，并重命名为main.py，然后reset或者重新上电。

如果不清楚K230如何烧入程序，请先看K230的快速入门章节教程。

4.掰动K230模块支架到合适的角度，接上电源。

5.当K230初始化成功后，会开始识别车牌，等待三秒后，小车开始追踪车牌。

注意：

如果对追踪的效果不满意，可以修改ProtocolHub工程中app_motor.c文件的PID值。

```
#define KP (12)
#define KI (0)
#define KD (0.1)
```

4.代码解析

- main.c

```
#define MOTOR_TYPE 5    //1:520电机 2:310电机 3:测速码盘TT电机 4:TT直流减速电机 5:L型520电机
                        //1:520 motor 2:310 motor 3:speed code disc TT motor 4:TT DC reduction motor 5:L type 520 motor

//修改当前要运行的颜色案例  Modify the color case currently being run
ColorMode color_mode = 1;  //1:巡线、2:跟踪  1: Patrol, 2: Tracking

//选择是否在初始化完成后，小车自主向前走，0为否，1为是(自主避让案例使用)
//Select whether the car moves forward autonomously after initialization is completed, 0 for no, 1 for yes (used in autonomous avoidance cases)
int Car_Auto_Drive = 0;

int main(void)
{
    USART_Init();

     //使能DMA通道，用于加快与四路电机驱动板串口的通讯  Enable DMA channel to speed up communication with the serial port of the four-way motor driver board
    NVIC_ClearPendingIRQ(UART_1_INST_INT_IRQN);
    DL_DMA_enableChannel(DMA, DMA_CH0_CHAN_ID);
    NVIC_EnableIRQ(UART_1_INST_INT_IRQN);

    //开启与K230串口通信的中断    Enable interrupts for communication with the K230 serial port
    NVIC_ClearPendingIRQ(UART_3_INST_INT_IRQN);
    NVIC_EnableIRQ(UART_3_INST_INT_IRQN);

    //设置电机类型    Set motor type
    Set_Motor(MOTOR_TYPE);


    if(Car_Auto_Drive)//K230上电初始化需要时间，等待初始化完成之后再开始前进。    It takes time for K230 to power on and initialize. Wait until the initialization is complete before moving forward.
    {

delay_ms(1000);delay_ms(1000);delay_ms(1000);delay_ms(1000);delay_ms(1000);delay_ms(1000);delay_ms(1000);delay_ms(1000);delay_ms(1000);
    }
    printf("Init Succeed\r\n");
    while(1)
    {
        Pto_Loop();
        delay_ms(10);
    }
}
```

MOTOR_TYPE：用于设置使用的电机类型，根据自己现使用的电机对照着注释修改对应的数字。

color_mode：要运行视觉巡线案例的时候使用1，如果是颜色跟踪案例的时候使用2

Car_Auto_Drive：控制小车在初始化后自动前进，默认为0，不自动前进。只有使用自主避让案例的时候需要开启

USART_Init：系统、串口的初始化函数。

Set_Motor：与四路电机驱动板通信，设置电机相关参数

Pto_Loop：持续处理接收到的K230的消息。

- yb_protocol.c

```
/**
 * @Brief: 数据分析 Data analysis
 * @Note:
 * @Parm: 传入接受到的一个数据帧和长度    Pass in a received data frame and length
 * @Retval:
 */
void Pto_Data_Parse(uint8_t *data_buf, uint8_t num)
{
    uint8_t pto_head = data_buf[0];
    uint8_t pto_tail = data_buf[num-1];

    //校验头尾  Check head and tail
    if (!(pto_head == PTO_HEAD && pto_tail == PTO_TAIL))
    {
        //DEBUG_PRINT("pto error:pto_head=0x%02x , pto_tail=0x%02x\n", pto_head,
pto_tail);
        return;
    }

    uint8_t data_index = 1;
    uint8_t field_index[PTO_BUF_LEN_MAX] = {0};
    int i = 0;
    int values[PTO_BUF_LEN_MAX] = {0};
    char msgs[][PTO_BUF_LEN_MAX] = {0};

    //分割字段  Split Field
    for (i = 1; i < num-1; i++)
    {
        if (data_buf[i] == ',')
        {
            data_buf[i] = 0;
            field_index[data_index] = i;
            data_index++;
        }
    }

    //解析长度与功能ID  Parsing length and function ID
    for (i = 0; i < 2; i++)
    {
        values[i] = Pto_Char_To_Int((char*)data_buf+field_index[i]+1);
```

```
    }

    uint8_t pto_len = values[0];
    uint8_t pto_id = values[1];


 ParseCommonFields(pto_id,data_buf,pto_len,field_index,data_index,values,msgs);
}

//获取数据函数  Get data function
void ParseCommonFields(
    int func_index,       // 功能索引（对应func_table中的位置） Function index (corresponding to the position in func_table)
    uint8_t *data_buf,    // 原始数据缓冲区 Raw data buffer
    uint8_t num,          // 数据总长度（含头尾）   Total length of data (including head and tail)
    uint8_t *field_index, // 字段分隔符位置数组（逗号的位置）   Array of field separator positions (positions of commas)
    uint8_t data_index,   // 总字段数（如数据有6个字段）    Total number of fields (if the data has 6 fields)
    int* values,          // 存储整数字段的数组 Array to store integer fields
    char msgs[][PTO_BUF_LEN_MAX]        // 存储字符串的指针数组 Array of pointers to store strings
){
    const FuncDesc* desc = &func_table[func_index-1];

    if(func_index != desc->id)
    {
        DEBUG_PRINT("Unsupported protocol ID: %d, Chosen ID:%d\n", func_index,desc->id);
        return;
    }

    for (int i = 0; i < desc->field_count; i++)
    {
        const FieldMeta* meta = &desc->fields[i];
        uint8_t pos = meta->pos;

        if (meta->type == FIELD_STRING) {
            int start = field_index[pos] + 1;
            int end = (pos+1 < data_index) ? field_index[pos+1] : (num-1);
            int len = end - start;

            len = (len < PTO_BUF_LEN_MAX-1) ? len : PTO_BUF_LEN_MAX-1;
            memcpy(msgs[i], data_buf+start, len);
            msgs[i][len] = '\0';
        } else {
            values[pos] = Pto_Char_To_Int((char*)data_buf + field_index[pos] +1);
        }
    }
    desc->handler(values, msgs);
}
```

将接收到的符合协议的数据进行解析，获取数据中的功能ID号、携带的数据信息，并执行对应功能的处理函数。本工程使用元数据解析的方法。核心思想是将协议格式的描述（字段位置、类型等）与解析逻辑分离，通过数据结构（如结构体数组）定义协议格式，解析函数根据这些描述自动处理数据。

协议中含有不同类型的数据也可以解析出来，想了解实现细节可以自行研究，不作过多的解答。此函数已适配了本店所有的K230通讯协议，不用完全理解也可以正常使用。

- bsp_motor_iic.c

```
//只能控制带编码器类型的电机 Can only control motors with encoders
//传入参数:4个电机的速度      Input parameters: speed of 4 motors
void control_speed(int16_t m1,int16_t m2 ,int16_t m3,int16_t m4)
{
    static uint8_t speed[8];

    speed[0] = (m1>>8)&0xff;
    speed[1] = (m1)&0xff;

    speed[2] = (m2>>8)&0xff;
    speed[3] = (m2)&0xff;

    speed[4] = (m3>>8)&0xff;
    speed[5] = (m3)&0xff;

    speed[6] = (m4>>8)&0xff;
    speed[7] = (m4)&0xff;

    i2cWrite(Motor_model_ADDR,SPEED_Control_REG,8,speed);

}

//控制带编码器类型的电机   Control the motor with encoder type
//传入参数:4个电机的pwm PWM of 4 motors
//此函数可以结合实时编码器的数据，来实现control_speed的功能   This function can combine the data of real-time encoder to realize the function of control_speed
void control_pwm(int16_t m1,int16_t m2 ,int16_t m3,int16_t m4)
{
    static uint8_t pwm[8];

    pwm[0] = (m1>>8)&0xff;
    pwm[1] = (m1)&0xff;

    pwm[2] = (m2>>8)&0xff;
    pwm[3] = (m2)&0xff;

    pwm[4] = (m3>>8)&0xff;
    pwm[5] = (m3)&0xff;

    pwm[6] = (m4>>8)&0xff;
    pwm[7] = (m4)&0xff;

    i2cWrite(Motor_model_ADDR,PWM_Control_REG,8,pwm);
```

```
}
```

`control_speed`：控制电机的速度。它将传入的四个电机速度值拆分成高低字节，然后通过I2C协议发送给电机控制器来设置电机的速度。

`control_pwm`：控制电机的PWM（脉宽调制）值。它将传入的四个电机的PWM值拆分成高低字节，通过I2C协议发送给电机控制器来控制电机的转速和方向。

- app_motor.c

```
//控制小车运行，输入的参数分别为x、y、z三个轴上的速度值
//Control the movement of the car. The input parameters are the speed values on
the x, y, and z axes.
void Motion_Car_Control(int16_t V_x, int16_t V_y, int16_t V_z)
{
    float robot_APB = Motion_Get_APB();
    speed_lr = 0;
    speed_fb = V_x;
    speed_spin = (V_z / 1000.0f) * robot_APB;
    if (V_x == 0 && V_y == 0 && V_z == 0)
    {
        control_pwm(0,0,0,0);
        return;
    }

    speed_L1_setup = speed_fb - speed_spin;
    speed_L2_setup = speed_fb - speed_spin;
    speed_R1_setup = speed_fb  + speed_spin;
    speed_R2_setup = speed_fb  + speed_spin;

    if (speed_L1_setup > 1000) speed_L1_setup = 1000;
    if (speed_L1_setup < -1000) speed_L1_setup = -1000;
    if (speed_L2_setup > 1000) speed_L2_setup = 1000;
    if (speed_L2_setup < -1000) speed_L2_setup = -1000;
    if (speed_R1_setup > 1000) speed_R1_setup = 1000;
    if (speed_R1_setup < -1000) speed_R1_setup = -1000;
    if (speed_R2_setup > 1000) speed_R2_setup = 1000;
    if (speed_R2_setup < -1000) speed_R2_setup = -1000;

    //DEBUG_PRINT("L1:%d, L2:%d, L3:%d,
L4:%d\r\n",speed_L1_setup,speed_L2_setup,speed_R1_setup,speed_R2_setup);
    //DEBUG_PRINT("\r\n");
    if(PWM_Car_Flag)//如果小车是无编码器类型的，就直接控制pwm，否则控制速度 If the car is
encoderless, it controls PWM directly, otherwise it controls the speed
    {
        control_pwm(speed_L1_setup, speed_L2_setup, speed_R1_setup,
speed_R2_setup);
    }
    else
    {
        control_speed(speed_L1_setup, speed_L2_setup, speed_R1_setup,
speed_R2_setup);
    }
}
```

```
void Licence_Track(int x1,int y1,int x2,int y2,int x3)
{
    if(!Wait_Flag)
    {
        delay_ms(1000);delay_ms(1000);delay_ms(1000);//delay 3s
        Wait_Flag = 1;
        return;
    }

    if(Wait_Flag && !SetTarget_Flag)
    {
        SetTarget_Flag = 1;
        target_area = (x3-x1)*(y2-y1)/100;
    }

    int now_area = (x3-x1)*(y2-y1)/100;

    int16_t err = 240 - x1;
    DEBUG_PRINT("x1:%d, CENTER:%d\r\n", x1, 240);
    DEBUG_PRINT("err:%d\r\n", err);
    if(err<5 && err>-5)err=0;

    pid_output = PID_Calc(err);
    DEBUG_PRINT("pid_output:%d\r\n", pid_output);
    if(pid_output>500) pid_output=500;
    if(pid_output<-500) pid_output=-500;

    DEBUG_PRINT("\r\n");

    int16_t err_area = target_area - now_area;
    DEBUG_PRINT("now_area:%d, target_area:%d\r\n", now_area, target_area);
    DEBUG_PRINT("err_area:%d\r\n", err_area);

    if(err_area<30 && err_area>-30)err_area=0;
    pid_output1 = PID_Calc(err_area);
    DEBUG_PRINT("pid_output1:%d\r\n", pid_output1);
    if(pid_output1>300) pid_output1=300;
    if(pid_output1<-300) pid_output1=-300;

    DEBUG_PRINT("\r\n");

    Motion_Car_Control(pid_output1, 0, pid_output);
}
```

Motion_Car_Control：用于控制小车的函数，输入的参数值会用于控制带编码器电机的速度，无编码器电机则会控制pwm值。

Licence_Track：识别到车牌的三秒后，记录当前车牌识别框的面积大小，然后计算x轴和面积的偏差值，控制小车减少这些偏差，从而实现追踪的效果，让识别框一直处于屏幕中间位置。

5.实验现象

打开电源开关，等待系统初始化完成后，屏幕会显示摄像头画面，并且会框起车牌，等待三秒后，小车会随着车牌的移动而移动


鲁A·66666